Documentation de conception

Projet DeCa

23 janvier 2026

Table des matières

Partie B : Vérification contextuelle orientée objet **3**

1 Objectif et périmètre de la partie B **3**

2 Organisation générale : passes et responsabilités **3**
2.1 Passe 1 – Déclaration des classes (structure & héritage) 3
2.2 Passe 2 – Déclaration des champs et méthodes 3
2.3 Passe 3 – Vérification des corps (instructions & expressions) 4

3 Architecture logicielle : classes principales et dépendances **4**
3.1 Packages impliqués 4
3.2 Vue par responsabilités 4

4 Environnements et définitions : modèle retenu **5**
4.1 Environnements (EnvironmentExp) : dictionnaire chaîné 5
4.2 Décoration par définitions 5

5 Règles gérées en partie B **5**
5.1 Héritage et résolution des classes 5
5.2 Déclaration des champs 5
5.3 Déclaration et redéfinition des méthodes 6
5.4 Visibilité protected 6
5.5 Expressions objet this, sélection, appel, cast 6

Partie C : Génération de code **7**

ETAPE C – Documentation de conception **7**

6 [GenCode] Génération de code : Partie C (sans objet) et Partie Objet **7**
6.1 Partie C : organisation en 2 passes 7
6.2 Décision centrale : gestion de registres et convention d'écriture 7
6.2.1 Un seul nœud "parent" teste `hasNext()` 7
6.2.2 Feuilles : elles allouent et chargent 7
6.3 Spill : algorithme factorisé dans `AbstractBinaryExpr` 7
6.4 Remarques sur `Assign` et `Identifier` (sans entrer dans les détails) 8
6.5 Choix de sûreté : sauvegarde systématique des registres dans une méthode 8

6.6 Constant folding : tentative puis abandon (robustesse et erreurs) 8
6.7 Partie Objet : algorithme de construction des vtables 9
6.7.1 Principe général 9
6.7.2 Récupération récursive des étiquettes : `getRecursiveMethodLabels` . . . 9
6.7.3 Génération assembleur : `codeGenTableConstruction` 9
6.8 Conclusion : robustesse, sécurité et qualité des erreurs 9

7 [GenCode] Architecture : classes clés et dépendances 9
7.1 Vue d'ensemble (rôles) 10
7.2 Sans objet : classes AST et dépendances principales 10
7.3 Objet : classes AST et dépendances principales 10
7.4 Dépendances clés (résumé par flèches) 11
7.5 Conclusion 11

Partie B : Vérification contextuelle orientée objet

1 Objectif et périmètre de la partie B

La vérification contextuelle garantit la cohérence sémantique d'un programme Deca après l'analyse syntaxique. La partie B (objets) étend cette vérification à :

- la déclaration des classes et l'héritage ;
- la déclaration et la cohérence des champs et méthodes ;
- les règles de redéfinition (override) ;
- la visibilité `protected` et les règles d'accès ;
- les expressions objets `this`, sélection de champ, appel de méthode, cast ;
- la décoration de l'AST par des informations contextuelles (Type, Definition) nécessaires à la génération de code.

Sortie attendue : un AST décoré (types + définitions) ou une erreur contextuelle explicite et localisée.

2 Organisation générale : passes et responsabilités

La vérification contextuelle est organisée en trois passes afin de résoudre progressivement les dépendances (héritage, signatures, corps des méthodes).

2.1 Passe 1 – Déclaration des classes (structure & héritage)

But : enregistrer toutes les classes avant de vérifier leur contenu.

Responsabilités :

- Créer une `ClassDefinition` pour chaque classe déclarée.
- Résoudre et valider le lien d'héritage (super-classe).
- Détecter/empêcher : redéfinition de classe, héritage depuis un type non-classe, héritage incohérent...

Initialisations nécessaires :

- Mise en place des types prédéfinis (ex. `int`, `float`, `boolean`, `string`, `void`, `null`...).
- Déclaration de la classe racine `Object` et de son environnement de membres.

Justification : La séparation « classe connue » / « contenu connu » évite les problèmes de références croisées entre classes (ex. une classe A utilise B avant l'analyse de B).

2.2 Passe 2 – Déclaration des champs et méthodes

But : construire l'API des classes (membres) sans analyser les corps.

Responsabilités :

- **Déclarer les champs :** création de `FieldDefinition`, vérification des doublons, validité des types, attribution d'un index (offset logique).
- **Déclarer les méthodes :** création de `MethodDefinition` (retour, signature, index), vérification des redéfinitions et cohérence avec la super-classe.
- **Mettre à jour pour chaque `ClassDefinition` :**
  - compteurs `numberOfFields` et `numberOfMethods`,

- environnement des membres `EnvironmentExp`.

Justification : Les corps de méthodes dépendent de la connaissance complète des signatures et membres (appel de méthode, accès champ, types de retour). Ces informations doivent donc être stabilisées avant la passe 3.

2.3 Passe 3 – Vérification des corps (instructions & expressions)

But : typer finement toutes les expressions/instructions et décorer l'AST.

Responsabilités :

- **Vérifier les corps de méthodes :** création d'un environnement local (paramètres + variables), vérification des instructions, vérification des `return`.
- **Vérifier les expressions objet :** `this`, sélection, appel, cast.
- Appliquer les règles de visibilité (`protected`) lors de la résolution des membres.
- **Décorer les nœuds AST avec :**
  - le `Type` de chaque expression,
  - la `Definition` associée à chaque identificateur (champ, méthode, variable, paramètre...).

3 Architecture logicielle : classes principales et dépendances

3.1 Packages impliqués

- `fr.ensimag.deca.tree` : nœuds de l'AST (classes, champs, méthodes, expressions).
- `fr.ensimag.deca.context` : types, définitions, environnements, compatibilités.
- `fr.ensimag.deca.tools` : table des symboles, gestion des localisations, erreurs.

3.2 Vue par responsabilités

Nœuds AST (tree) : Chaque nœud responsable d'une vérification contextuelle effectue une méthode de vérification (ex. `verify...`) :

- `Program` orchestre les passes.
- `ListDeclClass` itère sur les classes.
- `DeclClass` gère l'héritage, les membres, puis les corps.
- `DeclField` et `DeclMethod` gèrent la déclaration des champs/méthodes.
- Les expressions objet (ex. sélection, appel, cast, `this`, identificateurs) gèrent le typage et la résolution de définitions.

Contexte (context) :

- `Type` : représentation et relations du système de types.
- `Definition` et sous-classes : `ClassDefinition`, `FieldDefinition`, `MethodDefinition`, `VariableDefinition`, `ParamDefinition`.
- `EnvironmentExp` : environnement (dictionnaire) chaîné pour la résolution des noms.
- `Signature` : liste typée des paramètres d'une méthode.
- `ClassType` : type associé à une classe et lié à sa `ClassDefinition`.

Dépendances clés : Les nœuds `tree` dépendent de `context` pour calculer les types, créer/consulter les définitions et effectuer les résolutions via les environnements.

4 Environnements et définitions : modèle retenu

4.1 Environnements (EnvironmentExp) : dictionnaire chaîné

Un environnement chaîné (avec parent) est utilisé pour modéliser :

- la portée lexicale (variables locales) ;
- l'héritage (membres hérités accessibles via le parent) ;
- la résolution uniforme des symboles (recherche du nom en remontant les parents).

Conception pour l'héritage : Pour une classe C `extends` B :

- `envMembers(C)` contient les membres déclarés dans C,
- son parent correspond à `envMembers(B)`.

Justification : Ce schéma unifie les mécanismes de lookup (membres et variables), réduit les cas particuliers et simplifie la maintenance. Il impose néanmoins de distinguer clairement doublons locaux et membres hérités (override autorisé pour les méthodes selon règles, pas forcément pour les champs).

4.2 Décoration par définitions

Chaque identificateur est relié à une `Definition` :

- champ → `FieldDefinition` (type, visibilité, index, classe porteuse),
- méthode → `MethodDefinition` (retour, signature, index),
- variable locale → `VariableDefinition`,
- paramètre → `ParamDefinition`,
- classe → `ClassDefinition`.

Justification : Cette décoration rend possible une génération de code fiable (offset/index, accès champ, appel dynamique) et améliore la précision des diagnostics.

5 Règles gérées en partie B

5.1 Héritage et résolution des classes

- Une classe peut étendre une autre classe existante.
- La super-classe doit être un type classe (et non un type primitif).
- `Object` est la racine commune.
- **Optionnel :** détection de cycles d'héritage par remontée des super-classes et vérification d'absence de retour sur la classe courante.

5.2 Déclaration des champs

Un champ est caractérisé par :

- un type,
- un nom,
- une visibilité (ex. `protected` ou défaut),
- une `FieldDefinition` et un index (offset logique).

Contraintes usuelles :

- Interdiction de déclarer deux champs de même nom dans une même classe.
- Interdiction de type invalide (ex. `void`).

Index / offset logique : Les champs hérités conservent leurs positions, les champs nouveaux sont ajoutés après ceux de la super-classe, afin d'obtenir un layout compatible avec l'héritage.

5.3 Déclaration et redéfinition des méthodes

Une méthode est décrite par :

- nom,
- type retour,
- signature (types des paramètres),
- index (position dans la table des méthodes / vtable),
- `MethodDefinition`.

Override (redéfinition) : Quand une méthode existe déjà dans la super-classe :

- vérification de compatibilité de signature (même arité, types compatibles selon la règle retenue),
- vérification de compatibilité du type de retour (identique ou covariant selon règle),
- conservation de l'index de la méthode héritée.

Quand la méthode n'existe pas dans la super-classe :

- attribution d'un nouvel index et incrément de `numberOfMethods`.

Justification : Conserver l'index en override est indispensable pour l'appel dynamique : un appel via la vtable doit rediriger vers la bonne implémentation.

5.4 Visibilité protected

La visibilité `protected` restreint certains accès même si le membre existe. Une règle classique :

- accès autorisé depuis la classe courante et ses sous-classes,
- avec une contrainte sur le receveur (type statique de l'objet) selon la règle exacte.

Choix de conception : La vérification `protected` se fait lors de la résolution d'un accès membre (sélection/appel), car on connaît alors :

- la classe courante (`currentClass`),
- le type statique du receveur (`obj`).

5.5 Expressions objet this, sélection, appel, cast

this

- valide uniquement si l'on est dans un contexte de classe,
- son type est celui de la classe courante.

Sélection `obj.field`

- `obj` doit être de type classe,
- `field` doit exister (classe + héritage),
- application des règles `protected`,
- le type de l'expression est le type du champ.

Appel `obj.m(args...)`

- `obj` doit être de type classe,
- la méthode doit exister et être accessible,
- vérification du nombre d'arguments et compatibilité type-à-type,
- application des règles `protected`,
- le type de l'expression est le type de retour de la méthode.

Cast `(A) expr`

- cast vers une super-classe : généralement accepté statiquement,
- cast vers une sous-classe : accepté statiquement mais potentiellement invalide à l'exécution,
- cast entre classes non liées rejeté contextuellement,
- `null` est généralement compatible avec tout type classe.

Partie C : Génération de code

ETAPE C – Documentation de conception

6 [GenCode] Génération de code : Partie C (sans objet) et Partie Objet

Cette section décrit **uniquement** la génération de code (Partie C), en distinguant :

- **Sans objet** : expressions/instructions, registres, spill.
- **Objet** : classes, *vtables*, accès champ, appels de méthodes, corps de méthodes.

Nous restons au niveau **architecture** + **principes**, sans listings détaillés.

6.1 Partie C : organisation en 2 passes

Passe 1 : construction des tables des méthodes (vtables)

On parcourt la liste des classes afin de générer le code de **construction des vtables** (structures globales en mémoire). Cette passe produit uniquement les tables (pas les corps de méthodes).

Passe 2 : génération du main et des corps de méthodes

On génère ensuite :

- le **main** : déclarations puis instructions ;
- les **corps de méthodes** : prologue/épilogue, variables locales, instructions, retour.

6.2 Décision centrale : gestion de registres et convention d'écriture

6.2.1 Un seul nœud "parent" teste `hasNext()`

Dans notre projet, **seul** `AbstractBinaryExpr` décide explicitement s'il reste un registre libre via `hasNext()`. C'est le point unique où l'on choisit entre :

- **voie registre** (deux opérandes en registres),
- **voie spill** (pile) quand les registres sont saturés.

Tous les autres nœuds (ex : `Assign`, `AppelMethode`, `AccesChamp`, `IfThenElse`, `While`) **réutilisent le registre courant** où le fils a déposé son résultat, puis effectuent souvent un `freeRegister()` quand un registre temporaire n'est plus nécessaire.

6.2.2 Feuilles : elles allouent et chargent

Les **feuilles** (littéraux et identifiants) sont responsables de :

- **allouer** un registre via `allocateRegister()` ;
- **charger** la valeur dans ce registre (typiquement via `LOAD`).

Exemples de feuilles : `IntLiteral`, `FloatLiteral`, `BooleanLiteral`, `Identifier` (et côté objet : `This`, `Null`, `New`).

6.3 Spill : algorithme factorisé dans `AbstractBinaryExpr`

`AbstractBinaryExpr` factorise les opérations binaires arithmétiques (hors booléens dans notre structuration). L'algorithme est le suivant :

- Calcul de l'opérande gauche (résultat dans le registre courant).

- **Si** `hasNext()` : calcul de la droite dans un nouveau registre, exécution de l'opération, puis **libération** du registre de droite.
- **Sinon** : sauvegarde temporaire sur la pile (`PUSH`), calcul de la droite, restauration, opération, puis nettoyage de pile (par exemple `SUBSP`).

Ce choix unique garantit une **correction globale** des expressions complexes même avec `-r` petit.

6.4 Remarques sur `Assign` et `Identifier` (sans entrer dans les détails)

`Identifier`

`Identifier` est une feuille et parent aussi : elle alloue un registre et charge la valeur depuis l'opérande associé à la définition. En partie objet, elle gère aussi le cas des **champs** (chargement via `this`).

`Assign`

`Assign` évalue d'abord la partie droite (résultat en registre courant), puis effectue un `STORE` vers :

- une variable (adresse directe), (ex : x=3)
- ou un champ (adresse relative à l'objet, avec vérification `null` si nécessaire), (ex : c.x=1).

Dans tous les cas, `Assign` réutilise le registre produit par ses fils et libère les temporaires quand c'est possible. **Remarque :** `Assign` ne libère pas le registre résultat, afin de supporter les expressions chaînées du type `x = y = 3`. La libération finale est donc réalisée par le parent (par exemple `ListInst`), une fois que la valeur n'est plus nécessaire.

6.5 Choix de sûreté : sauvegarde systématique des registres dans une méthode

Dans les corps de méthodes, nous avons privilégié une approche **sûre et simple** :

- vérification de pile en entrée (`TSTO` puis `BOV`) ;
- sauvegarde de tous les registres `R2..R_max` (`PUSH`) ;
- restauration symétrique en sortie (`POP`) puis `RTS`.

Justification : une optimisation plus fine (ne sauver que les registres réellement utilisés) demande une analyse supplémentaire (vivacité, convention stricte d'appel, etc.) et augmente le risque de bugs. Étant donné le contexte projet, nous avons choisi la **robustesse** et la **correction**.

6.6 Constant folding : tentative puis abandon (robustesse et erreurs)

Nous avions ajouté une optimisation de **constant folding** (simplification des expressions constantes). Elle a été **abandonnée** car :

- elle introduisait des cas limites sur des expressions complexes (interaction avec la saturation registres/spill) ;
- elle compliquait la gestion cohérente des **messages d'erreur** (overflow, division par zéro, etc.) ;
- elle demandait davantage d'invariants pour garantir la conformité au comportement d'exécution.

Le choix final a été : **priorité à la correction et à des erreurs propres**, plutôt qu'une optimisation risquée.

6.7 Partie Objet : algorithme de construction des vtables

6.7.1 Principe général

Chaque vtable est construite en mémoire globale (base `GB`) et contient :

- un pointeur vers la vtable de la super-classe ;
- puis une liste d'étiquettes de méthodes (`code.C.m`) dans un ordre indexé.

La vtable racine est initialisée avec `code.Object.equals`, ce qui fixe naturellement l'index de `equals` dans toute la hiérarchie.

6.7.2 Récupération récursive des étiquettes : `getRecursiveMethodLabels`

La liste finale des labels d'une classe est construite en :

1. identifiant la classe parent dans la liste des classes ;
2. récupérant récursivement la vtable du parent ;
3. initialisant la racine avec `code.Object.equals` si aucun parent concret n'existe.

Puis, pour chaque méthode déclarée dans la classe courante :

- on construit son label `code.<Classe>.<methode>` ;
- on cherche si une méthode de même nom existe dans la liste héritée :
  - si oui (et si ce n'est pas `equals` selon notre règle), on **remplace** l'entrée → **override** ;
  - sinon, on **ajoute** en fin → **nouvelle méthode**.

Un override conserve ainsi l'**index** du parent, ce qui garantit la stabilité des appels dynamiques.

6.7.3 Génération assembleur : `codeGenTableConstruction`

Lors de la génération :

1. on stocke d'abord l'adresse de la vtable parent (via `LEA` puis `STORE`) ;
2. pour chaque label, on charge son adresse (`LOAD LabelOperand`) et on stocke dans la zone globale (`STORE`).

La fonction renvoie l'index global suivant, ce qui permet d'enchaîner proprement les vtables en mémoire.

6.8 Conclusion : robustesse, sécurité et qualité des erreurs

Le fil conducteur de notre implémentation est la **robustesse** :

- une stratégie de spill unique et factorisée (`AbstractBinaryExpr`) ;
- une génération objet structurée (vtables stables, override par remplacement, index conservés) ;
- une priorité donnée à la **correction** et à la **qualité des messages d'erreur** plutôt qu'à des optimisations risquées (abandon du constant folding, sauvegarde systématique des registres dans les méthodes).

Cela garantit un compilateur fiable en démonstration : comportements reproductibles, erreurs explicites (`null_pointer_error`, `stack_overflow_error`, `division_zero_error`, etc.) et exécution conforme.

7 [GenCode] Architecture : classes clés et dépendances

Cette partie liste les **classes importantes** impliquées dans la génération de code et leurs **dépendances**, au niveau « architecture » (pas de détails Javadoc / pas de listing).

7.1 Vue d'ensemble (rôles)

- `DecacCompiler` : point d'entrée de la génération, transporte l'état global (`IMAProgram`, tailles de pile, labels d'erreur, options, `RegisterAllocater`).
- `RegisterAllocater` : gestion linéaire des registres (registre courant, `allocate/free`, `hasNext`).
- `IMAProgram` + **instructions IMA** : cible de sortie ; `DecacCompiler.addInstruction()` délègue vers `IMAProgram`.

7.2 Sans objet : classes AST et dépendances principales

Hiérarchie AST (expressions / instructions)

- `AbstractExpr` → base des expressions, définit le point d'extension `codeGenInst()` et les variantes `codeGenPrint()`.
- `AbstractBinaryExpr` → **seul parent** qui teste `hasNext()` et gère le spill (pile).
- `Identifier`, `IntLiteral`, `FloatLiteral`, `BooleanLiteral` → **feuilles** : elles allouent un registre et y chargent leur valeur.
- `Assign` → réutilise le registre produit par le fils droit, puis `STORE` vers la destination.
- `AbstractInst` → base des instructions ; spécialisations typiques : `IfThenElse`, `While`, `Return`.

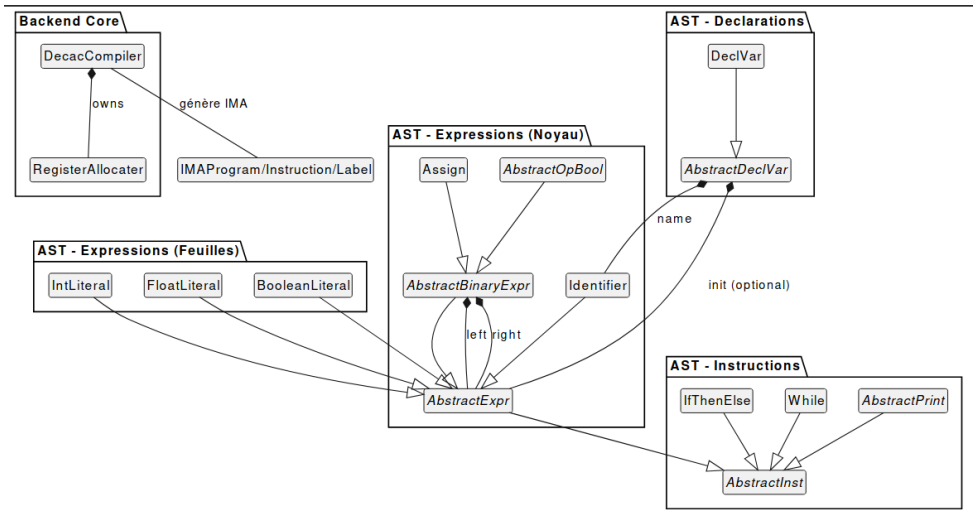

FIGURE 1 – Relations importantes entre les classes (partie sans objet)

Dépendances structurantes (sans objet)

- `AbstractBinaryExpr` → dépend de `DecacCompiler` pour accéder à `RegisterAllocater` et émettre des instructions IMA.
- Feuilles (`IntLiteral`, `Identifier`, ...) → dépendent de `RegisterAllocater` (allocation) et d'IMA (`LOAD`).
- `Assign` → dépend de l'évaluation de la `current_register` + d'IMA (`STORE`).

7.3 Objet : classes AST et dépendances principales

Programme / Classes

- `Program` → contient `ListDeclClass` + `Main`.
- `ListDeclClass` → orchestre la passe vtable puis la génération des corps de méthodes.
- `DeclClass` → porte l'algorithme de construction de vtable (récupération récursive + override/remplacement).

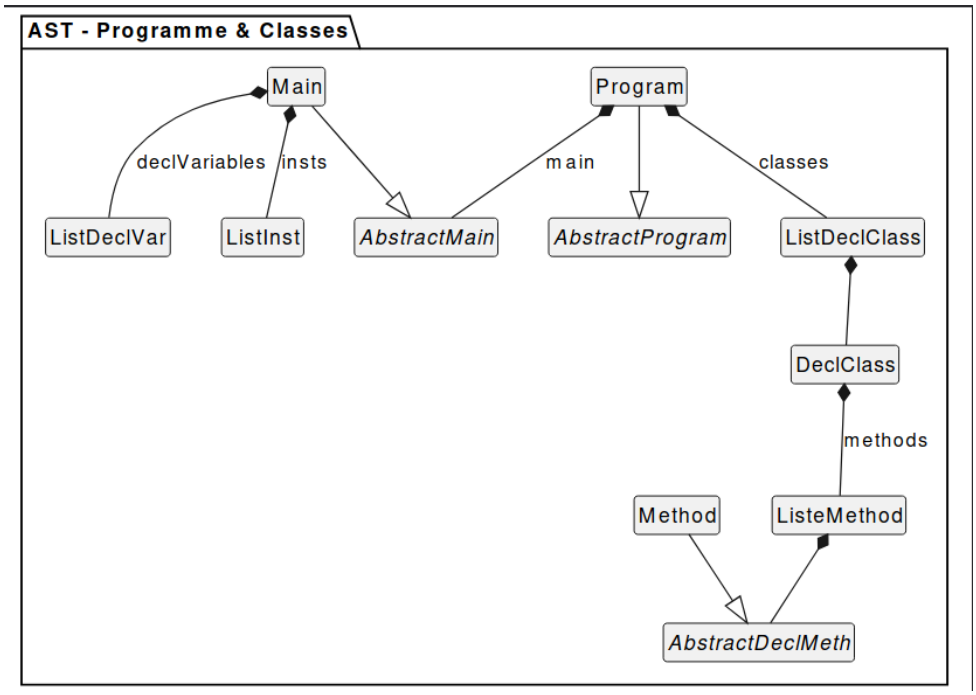

FIGURE 2 – Classes importantes de la partie objet

Méthodes

- `ListeMethod` → itère sur `AbstractDeclMeth`.
- `Method` → prologue/épilogue de méthode : `TSTO/BOV` + sauvegarde/restauration `R2..R_max`, gestion des locaux, label de fin.

Expressions objet

- `New` → allocation objet + initialisation, dépend d'IMA (tas) et des infos de classe (taille, vtable).
- `This` → feuille : charge `this` depuis l'activation record (ex : `-2(LB)`).
- `Null` → feuille : charge `null`.
- `AccesChamp` → `obj.codeGenInst()` puis `LOAD offset(obj)` avec test `null` si activé.
- `AppelMethode` → empile `this` + args, récupère la vtable, puis `BSR` à l'index de méthode.

7.4 Dépendances clés (résumé par flèches)

- `DecacCompiler` → `RegisterAllocater`
- `DecacCompiler` → `IMAProgram` (via `addInstruction/addLabel/addComment`)
- `AbstractExpr/AbstractInst` → `DecacCompiler` (émission IMA + accès aux options)
- `AbstractBinaryExpr` → `RegisterAllocater` (unique usage de `hasNext`)
- `Program` → `ListDeclClass`, `Main`
- `DeclClass` → construction vtable (labels) → écrit en mémoire globale (`GB`)

7.5 Conclusion

L'architecture de génération de code de notre compilateur est organisée autour d'un noyau minimal et robuste : `DecacCompiler` centralise l'état (programme IMA, options, tailles de pile, labels d'erreurs) et fournit une interface uniforme d'émission d'instructions, tandis que `RegisterAllocater` encapsule une politique simple d'allocation/libération.

Côté AST, nous avons volontairement concentré la complexité au bon endroit : `AbstractBinaryExpr` est le seul nœud parent qui vérifie `hasNext()` et gère le *spill* par pile, ce qui rend le reste des nœuds plus prévisibles. Les feuilles (`IntLiteral`, `Identifier`, `This`, `Null`, etc.) se contentent d'allouer un registre et d'y déposer leur résultat, puis les nœuds intermédiaires réutilisent ce registre et libèrent au besoin. Cette discipline limite les effets de bord et facilite le débogage.

Nous avons également privilégié une **factorisation claire** du code de génération, afin d'éviter la duplication et de garantir une cohérence globale : les comportements communs (schéma "calcul → registre courant → réutilisation/libération", génération standard des opérations binaires, points d'insertion des contrôles d'erreurs) sont regroupés dans des classes abstraites et des méthodes partagées, plutôt que recopiés dans chaque nœud. Cette organisation rend l'évolution du compilateur plus simple (ajout d'un nouvel opérateur ou d'une nouvelle feuille) et réduit les risques de divergences entre cas similaires.

Pour la partie objet, le choix d'une construction explicite des *vtables* (héritage récursif + remplacement des labels en cas d'override) fournit un mécanisme clair pour le dispatch dynamique. Enfin, nous avons privilégié la **sûreté et la correction** : tests de pile (`TSTO`/`BOV`), vérifications `null` si l'option `-n` n'est pas activée, prologue/épilogue de méthodes conservateurs (sauvegarde/restauration des registres) — même si cela coûte quelques instructions — afin de garantir un comportement stable et des erreurs d'exécution diagnostiquables. Cette approche donne un compilateur plus fiable à démontrer et plus simple à maintenir/faire évoluer.